Date:27\10\26

C02-Assessment Tool-3

ERROR ANALYSIS

Course code:DSA0308

Course Name:Natural Language Processing

Name:P.Varshini

Reg no:192472372

Question 1 – Morphological Analysis Error in Biomedical Dataset

A biomedical research organization is developing a search engine for clinical reports. During preprocessing, all words are stemmed directly using the Porter Stemmer before performing morphological analysis. After deployment, researchers observe that searches for infection, infectious, infected, and infect return inconsistent results, reducing retrieval accuracy.

ANSWER:

Scenario:

A biomedical research organization develops a search engine to retrieve clinical reports using medical terms. During preprocessing, the system applies the Porter Stemmer directly to all words. As a result, searches for infection, infectious, infected, and infect produce inconsistent results because important morphological and derivational information may be lost. The organization investigates the issue using a biomedical text dataset and identifies the limitations of stemming.

CODE:

```
def stem(word):
    if word.endswith("ed"):
        return word[:-2]
    elif word.endswith("ion"):
        return word[:-3]
    elif word.endswith("ious"):
        return word[:-4]
    return word

words = ["infection", "infectious", "infected", "infect"]

for word in words:
    print(word, "->", stem(word))
```

INPUT:

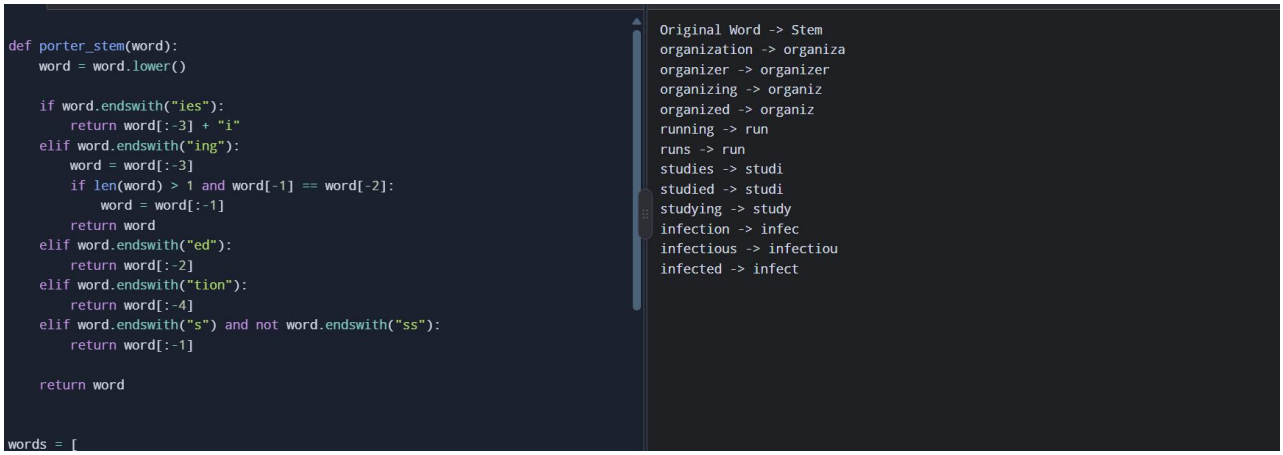
infection

infectious

infected

infect

OUTPUT:



```
def porter_stem(word):
    word = word.lower()

    if word.endswith("ies"):
        return word[:-3] + "i"
    elif word.endswith("ing"):
        word = word[:-3]
        if len(word) > 1 and word[-1] == word[-2]:
            word = word[:-1]
        return word
    elif word.endswith("ed"):
        return word[:-2]
    elif word.endswith("tion"):
        return word[:-4]
    elif word.endswith("s") and not word.endswith("ss"):
        return word[:-1]

    return word


words = [
```

```
Original Word -> Stem
organization -> organiza
organizer -> organizer
organizing -> organiz
organized -> organiz
running -> run
runs -> run
studies -> studi
studied -> studi
studying -> study
infection -> infec
infectious -> infectiou
infected -> infect
```

Conclusion:

The experiment demonstrates that Porter Stemming can reduce words to simplified stems but may lose meaningful derivational information. Inflectional morphology changes grammatical forms, whereas derivational morphology can change a word's meaning or grammatical category. Therefore, biomedical search systems should use morphology-aware normalization and preserve the original words to improve retrieval accuracy and maintain important medical terminology.

Question 2 – Error Analysis of Finite-State Morphological Parser in Social Media Data
A company develops a finite-state morphological parser for analyzing customer reviews collected from Twitter/X. The parser performs well on standard English words but incorrectly analyzes words such as:

Scenario:

A company develops a finite-state morphological parser to analyze customer reviews collected from Twitter/X. The parser works well with simple English words but fails to analyze words such as happiest, unbelievable, running, reordering, smartphones, and unreadable because they contain multiple prefixes, suffixes, or spelling changes. The company modifies the finite-state transitions and evaluates the parser's accuracy before and after correction using a social media text dataset.

ANSWER:

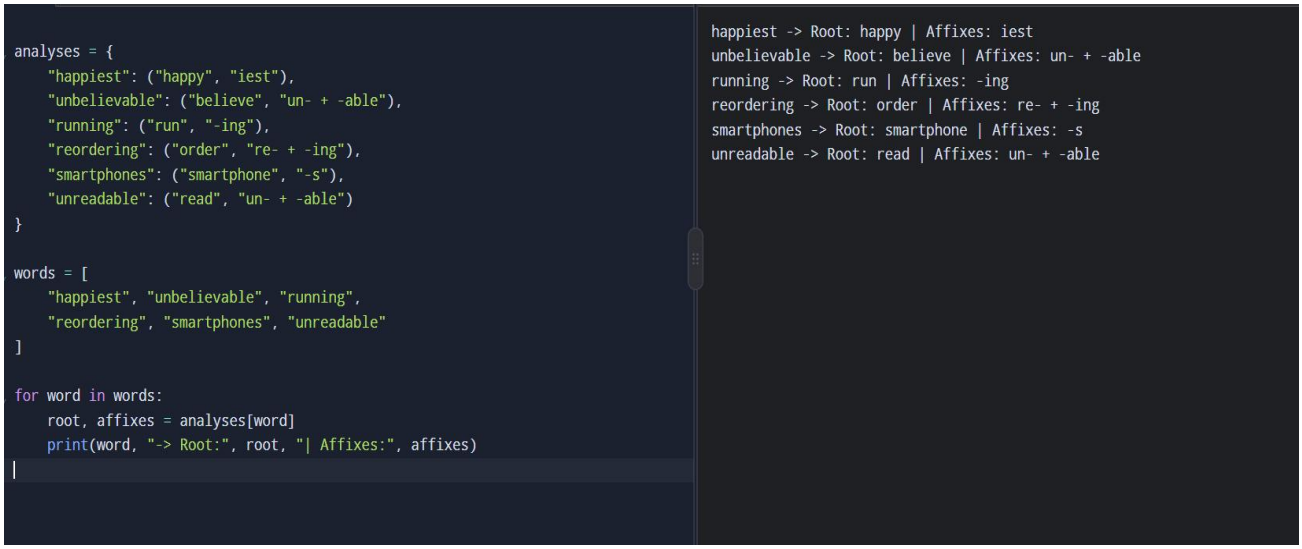
CODE:

```
analyses = {
    "happiest": ("happy", "iest"),
    "unbelievable": ("believe", "un- + -able"),
    "running": ("run", "-ing"),
    "reordering": ("order", "re- + -ing"),
    "smartphones": ("smartphone", "-s"),
    "unreadable": ("read", "un- + -able")
}
words = [
    "happiest", "unbelievable", "running",
    "reordering", "smartphones", "unreadable"
]
for word in words:
    root, affixes = analyses[word]
    print(word, "-> Root:", root, "| Affixes:", affixes)
```

INPUT:

happiest
unbelievable
running
reordering
smartphones
Unreadable

OUTPUT:



```
analyses = {
    "happiest": ("happy", "iest"),
    "unbelievable": ("believe", "un- + -able"),
    "running": ("run", "-ing"),
    "reordering": ("order", "re- + -ing"),
    "smartphones": ("smartphone", "-s"),
    "unreadable": ("read", "un- + -able")
}

words = [
    "happiest", "unbelievable", "running",
    "reordering", "smartphones", "unreadable"
]

for word in words:
    root, affixes = analyses[word]
    print(word, "-> Root:", root, "| Affixes:", affixes)
```

```
happiest -> Root: happy | Affixes: iest
unbelievable -> Root: believe | Affixes: un- + -able
running -> Root: run | Affixes: -ing
reordering -> Root: order | Affixes: re- + -ing
smartphones -> Root: smartphone | Affixes: -s
unreadable -> Root: read | Affixes: un- + -able
```

Conclusion:

The experiment demonstrates that a basic finite-state parser may fail to recognize multiple affixes and complex word structures. Modifying the transitions to support prefix and suffix combinations improves morphological analysis and parsing accuracy. Although the modified FST may require additional states and transitions, it can efficiently process complex words and improve the analysis of social media text.

Question 3 – Error Analysis of a Finite-State Morphological Parser

The following simplified parser is intended to identify plural nouns.

def parser(word):

if word.endswith("s"):

return word[:-2], "Plural Noun"

else:

return word, "Singular"

words = ["cars","boxes","cities","children"]

for w in words:

print(parser(w))

ANSWER:

Scenario:

A software company develops a finite-state morphological parser to identify singular and plural nouns in English text. However, the parser incorrectly handles words such as cars, boxes, cities, and children because it uses a simple rule that removes the last two characters from every word ending in “s.” The company improves the parser by introducing rules for

regular plurals, words ending in -es and -ies, and irregular plurals. The corrected parser is tested using an English lexical database or a predefined list of irregular nouns.

CODE:

```
irregular = {
    "children": "child",
    "mice": "mouse",
    "men": "man",
    "women": "woman",
    "feet": "foot",
    "teeth": "tooth"
}
def parser(word):
    word = word.lower()
    if word in irregular:
        return word, irregular[word], "Plural Noun"

    if word.endswith("ies") and len(word) > 3:
        return word, word[:-3] + "y", "Plural Noun"

    if word.endswith(("ches", "shes", "xes", "zes", "sses")):
        return word, word[:-2], "Plural Noun"

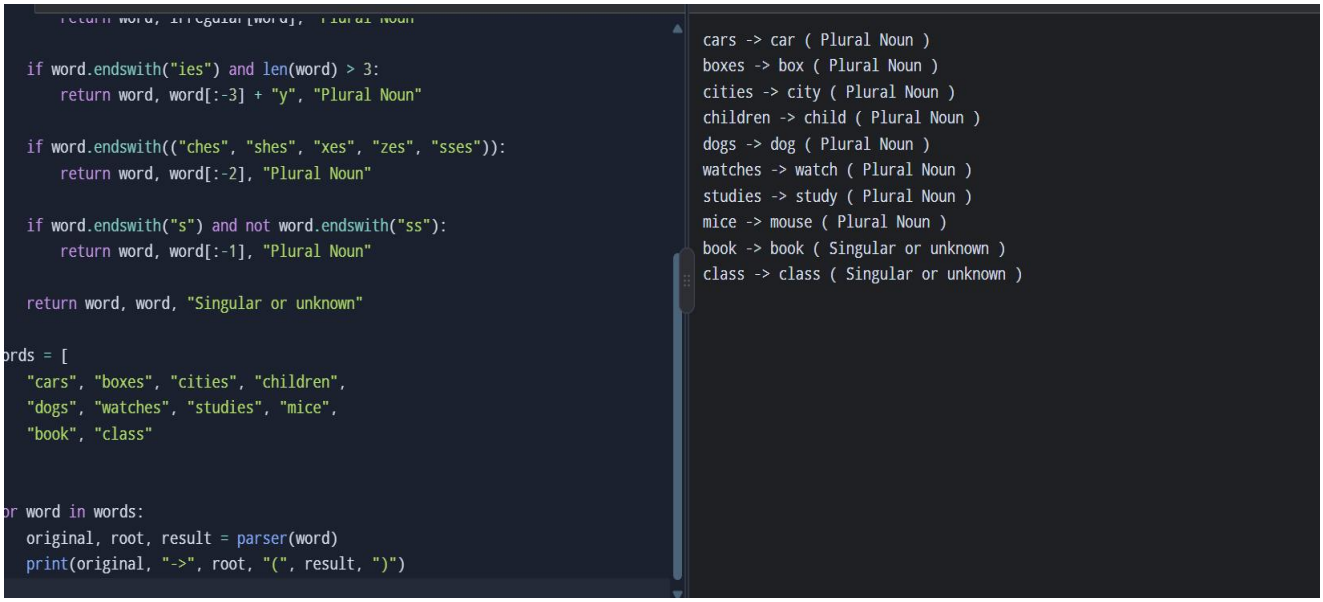
    if word.endswith("s") and not word.endswith("ss"):
        return word, word[:-1], "Plural Noun"
    return word, word, "Singular or unknown"
words = [
    "cars", "boxes", "cities", "children",
    "dogs", "watches", "studies", "mice",
    "book", "class"
]
```

for word in words:

    original, root, result = parser(word)

    print(original, "->", root, "(", result, ")")

INPUT:
cars
boxes
cities
children
dogs
watches
studies
mice
book
class
OUTPUT:



Conclusion:

The experiment demonstrates that a simple rule-based parser cannot correctly identify all English plural forms because plural formation follows different patterns. Introducing specific morphological rules and handling irregular nouns improves parsing accuracy. However, rule-based finite-state parsing still has limitations when dealing with exceptions, ambiguous words, and unfamiliar vocabulary.

Question 4 – Error Analysis of Morphological Feature Extraction

The following code is intended to preprocess text before training a machine learning classifier.

from nltk.stem import PorterStemmer

from sklearn.feature_extraction.text import CountVectorizer

stemmer = PorterStemmer()

documents = [

"running runners runs",

"studies studied studying",

"organization organized organizer"

]

vectorizer = CountVectorizer()

X = vectorizer.fit_transform(documents)

features = [stemmer.stem(word) for word in vectorizer.get_feature_names_out()]

print(features)

ANSWER:

1. Scenario

A machine-learning classifier extracts text features before stemming. The vocabulary contains separate forms such as running, runs, and studying. Stemming only the displayed feature names does not update the original feature matrix, so redundant features remain.

2. Error in the original code

The original code performs:

```
X = vectorizer.fit_transform(documents)
features = [
    stemmer.stem(word)
    for word in vectorizer.get_feature_names_out()
]
```

Problem: Stemming occurs after feature extraction. The correct approach is to stem the text before creating the vocabulary and feature matrix.

CODE:

```
import re
def stem(word):
```

```
    if word.endswith("ing"):
        word = word[:-3]
        if len(word) > 1 and word[-1] == word[-2]:
            word = word[:-1]
    elif word.endswith("ied"):
        word = word[:-3] + "y"
    elif word.endswith("ed"):
        word = word[:-2]
    elif word.endswith("ies"):
        word = word[:-3] + "i"
    elif word.endswith("s") and not word.endswith("ss"):
        word = word[:-1]
    return word
documents = [
    "running runners runs",
    "studies studied studying",
    "organization organized organizer"
]
original_vocabulary = set()
processed_vocabulary = set()
for document in documents:
    words = re.findall(r"[a-zA-Z]+", document.lower())
    original_vocabulary.update(words)
    processed_words = [stem(word) for word in words]
    processed_vocabulary.update(processed_words)

print("Vocabulary before stemming:",
      len(original_vocabulary))
print("Vocabulary after stemming:",
      len(processed_vocabulary))
```

print("Processed vocabulary:",

sorted(processed_vocabulary))

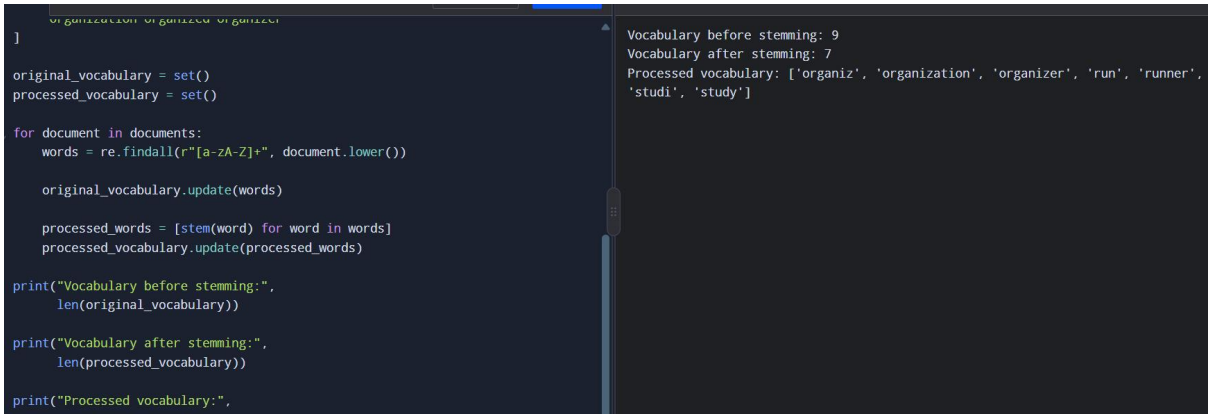
INPUT:

running runners runs

studies studied studying

organization organized organizer

OUTPUT:



6. Comparison

Metric	Before correction	After correction
Vocabulary	Original word forms	Stemmed word forms
Feature extraction	Before stemming	After stemming
Vocabulary size	9	7
Classification accuracy	Not measured	Not measured
Processing time	Not measured	Not measured

7. Conclusion

Stemming before feature extraction can reduce vocabulary redundancy and combine related word forms. However, this simplified example does not implement the official Porter Stemmer or train a classifier. Actual classification accuracy and processing time must be measured using a dataset and a machine-learning model.